

Week 5 : Payment Details

As The E-commerce company sells multiple product . we have stored order details and need to store payment details. The **Payment Details** section records all financial transactions linked to customer orders. It captures essential information such as payment mode, status, date, and amount. This ensures accuracy, transparency, and easy tracking of every transaction.

1.Create Payment Table

```
SQL> CREATE TABLE Payment(Payment_ID INT,Order_ID INT,Product_ID VARCHAR2(10),Product_Name VARCHAR2(15),Payment_Date DATE,Payment_Mode VARCHAR2(20),Payment_Status VARCHAR2(25),Transaction_Amount NUMBER(10,2));
Table created.
```

2.Define Relationships and Constraints

Relationships

- **One-to-Many (1:M):**
 - A single **Category** can have many **Products**.
 - One **Order** can include multiple **Products**.
- **One-to-One (1:1):**
 - Each **Payment** is linked to exactly one **Order**.
- **Many-to-Many (M:N):**
 - Customers and Products can be connected through **Orders** (a customer can buy many products, and a product can be bought by many customers).

Constraints

- ❖ **Primary Key (PK):** Ensures each record is unique.
- ❖ **Foreign Key (FK):** Maintains referential integrity between tables.
- ❖ **NOT NULL:** Prevents empty values in critical fields.
- ❖ **DEFAULT:** Provides automatic values when none are given.
- ❖ **CHECK:** Validates data based on conditions.

1.we add primary key to Order details Table

```
SQL> ALTER TABLE Payment
  2  ADD CONSTRAINT PK_PAYMENT PRIMARY KEY(PAYMENT_ID);

Table altered.
```

2.we added foregin key order id

```
SQL> ALTER TABLE Payment
  2  ADD CONSTRAINT FK_Order FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID);

Table altered.
```

Insert values in Payment Table

```
SQL> INSERT INTO Payment
  2  (Payment_ID, Order_ID, Product_ID, Product_Name, Payment_Date, Payment_Mode, Payment_Status, Transaction_Amount)
  3  VALUES
  4  (02, 1002, 'P00001', 'Ceramic Bowl',DATE'2026-08-27', 'Credit Card', ' Success', 250.00);

1 row created.

SQL> INSERT INTO Payment
  2  (Payment_ID, Order_ID, Product_ID, Product_Name, Payment_Date, Payment_Mode, Payment_Status, Transaction_Amount)
  3  VALUES
  4  (03, 1003, 'P00002', 'Coffee Cup',DATE'2026-10-09', 'Cash', 'Success', 600.00);

1 row created.

SQL> INSERT INTO Payment
  2  (Payment_ID, Order_ID, Product_ID, Product_Name, Payment_Date, Payment_Mode, Payment_Status, Transaction_Amount)
  3  VALUES
  4  (04, 1004, 'P00006', 'Ceramic Plate',DATE'2026-10-15', 'UPI', 'Success', 200.00);

1 row created.

SQL> INSERT INTO Payment
  2  (Payment_ID, Order_ID, Product_ID, Product_Name, Payment_Date, Payment_Mode, Payment_Status, Transaction_Amount)
  3  VALUES
  4  (05, 1005, 'P00003', 'Garder Planter',DATE'2026-10-29', 'Debit Card', 'Success', 1200.00);

1 row created.

SQL> INSERT INTO Payment
  2  (Payment_ID, Order_ID, Product_ID, Product_Name, Payment_Date, Payment_Mode, Payment_Status, Transaction_Amount)
  3  VALUES
  4  (06, 1006, 'P00005', 'Wall Hanging', DATE'2026-10-26', 'Net Banking', 'Success', 750.00);

1 row created.
```

Result

```
SQL> SELECT * FROM PAYMENT;
```

PAYMENT_ID	ORDER_ID	PRODUCT_ID	PRODUCT_NAME	PAYMENT_D	PAYMENT_MODE
Success	2	1002	P00001	Ceramic Bowl	27-AUG-26 Credit Card
			250		
Success	3	1003	P00002	Coffee Cup	09-OCT-26 Cash
			600		
Success	4	1004	P00006	Ceramic Plate	15-OCT-26 UPI
			200		

PAYMENT_ID	ORDER_ID	PRODUCT_ID	PRODUCT_NAME	PAYMENT_D	PAYMENT_MODE
Success	5	1005	P00003	Plant Pot	29-OCT-26 Debit Card
			1200		
Success	6	1006	P00005	Wall Hanging	26-OCT-26 Net Banking
			750		
Success	1	1001	P00004	Flower Vase	26-SEP-26 UPI
			550		

6 rows selected.

Report 1: Payment Method used by Customer

- ✓ Number of payment using UPI, cash, Net banking and card

```
SQL> SELECT COUNT(*) AS UPI_TRANSACTION
2 FROM Payment
3 WHERE Payment_Mode = 'UPI';
```

UPI_TRANSACTION

2

```
SQL> SELECT COUNT(*) AS CARD_TRANSACTION
2 FROM Payment
3 WHERE Payment_Mode = 'Credit Card';
```

CARD_TRANSACTION

1

```
SQL> SELECT COUNT(*) AS CARD_TRANSACTION
2 FROM Payment
3 WHERE Payment_Mode = 'Debit Card';
```

CARD_TRANSACTION

1

```
SQL> SELECT COUNT(*) AS ONLINE_TRANSACTION
2 FROM Payment
3 WHERE Payment_Mode = 'Net Banking';
```

ONLINE_TRANSACTION

1

Report 2 : Total Revenue

- Shows the **overall earnings** before deducting costs or expenses.
- Helps management evaluate **sales performance**.
- Useful for comparing revenue across **time periods, products, or payment modes**.

```
SQL> SELECT Payment_Mode,
2  SUM(Transaction_Amount) AS Total_Revenue
3  FROM Payment
4  WHERE Payment_Status = 'Success'
5  GROUP BY Payment_Mode
6  ORDER BY Total_Revenue DESC;
```

PAYMENT_MODE	TOTAL_REVENUE
-----	-----
Debit Card	1200
Net Banking	750
UPI	750
Cash	600

```
SQL> SELECT ROUND(AVG(TRANSACTION_AMOUNT),2)AS TOTAL
2  FROM PAYMENT
3  WHERE PAYMENT_STATUS='Success' ;
```

TOTAL

660

Report 3 : Customer Payment History

- The **Customer Payment History** report tracks all payments made by each customer over time.
- Helps businesses **monitor customer behavior** and payment patterns.
- Useful for identifying **loyal customers, delayed payments, or preferred payment methods**.
- Supports customer service teams in resolving disputes or verifying transactions.

```
SQL> SELECT
  2     O.ORDER_ID AS ORDER_ID,
  3     O.ORDER_DATE AS ORDER_DATE,    -- safer alias
  4     P.PAYMENT_DATE AS PAYMENT_DATE,
  5     P.PAYMENT_MODE AS PAYMENT_MODE,
  6     P.TRANSACTION_AMOUNT AS AMOUNT,
  7     P.PAYMENT_STATUS AS PAYMENT_STATUS
  8 FROM ORDERS O
  9 JOIN PAYMENT P
 10     ON O.ORDER_ID = P.ORDER_ID
 11 ORDER BY P.PAYMENT_DATE;
```

ORDER_ID	ORDER_DAT	PAYMENT_D	PAYMENT_MODE	AMOUNT
PAYMENT_STATUS				
1002	27-AUG-26	27-AUG-26	Credit Card	250
Success				
1001	28-SEP-26	26-SEP-26	UPI	550
Success				
1003	09-SEP-26	09-OCT-26	Cash	600
Success				
ORDER_ID	ORDER_DAT	PAYMENT_D	PAYMENT_MODE	AMOUNT
PAYMENT_STATUS				
1004	15-SEP-26	15-OCT-26	UPI	200
Success				
1006	30-SEP-26	26-OCT-26	Net Banking	750
Success				
1005	29-SEP-26	29-OCT-26	Debit Card	1200
Success				

6 rows selected.

The **Payment** table is designed to store and manage all financial transaction details related to customer orders